

53 New Integer Schemes for 3×3 Matrix Multiplication with 23 Products

Greg Sidebottom

Reproducible report — logic repo (<https://github.com/gsidebottom/logic>), matmul track, 2026-07-03 (commits 45e5acc..93347f2). All artifacts referenced live in this repository; every claim has a mechanical check listed in §4.

1 Summary

We report **53 new schemes for multiplying two 3×3 matrices with 23 multiplications, with coefficients in $\{-1, 0, +1\}$** — bilinear, so valid over any ring (commutativity of the entries is never used, which is what lets them recurse on block matrices), the same object class as Laderman’s 1976 scheme. Each scheme is:

1. **verified mod 2** against all 729 Brent equations,
2. **pairwise inequivalent** under the full de Groote symmetry group $(\text{GL}(3, 2)^3 \rtimes S_3$, exact witness-or-refutation checks),
3. **inequivalent to every published scheme**: all 17,376 schemes of the Kauers–Heule–Seidl (HKS) database *and* the four classics (Laderman 1976, Smirnov 2013, Oh–Kim–Moon 2013, Courtois–Bard–Hulme 2011),
4. **lifted to integer coefficients** in $\{-1, 0, +1\}$ and verified **exactly over $\mathbb{Z}$** against all 729 integer Brent equations.

Discovery cost: the 53 came out of one **6-minute single-threaded run** of a neighborhood-walk pipeline (138 raw finds $\rightarrow$ 53 survived full-database novelty checking), plus ~ 40 minutes of certification compute. (All timings in this report are from one machine: a Mac mini with an Apple M4 Pro — 10 performance + 4 efficiency cores — and 64 GB RAM, macOS.) For scale: the HKS local-search campaign reported “more than 13,000 mutually inequivalent” schemes for about 35 years of CPU time (SAT 2019, arXiv:1903.11391); the public database that grew out of it — our novelty reference — holds 17,376 (methods differ; see §7 caveats, including a normalized rate comparison).

The engine behind the discovery is a **native-ANF stochastic local search (SLS)**: the Brent system is kept as 729 cubic-XOR constraints over the 621 real variables instead of a $\sim 26,500$ -variable CNF, which extends the seeded-repair horizon by $\geq 5,000\times$ over yalsat-on-CNF and, with an exact GF(2) “tensor closure” move, solves 8/10 of the official HKS challenge-1 instances (yalsat’s published record: 5/10).

2 Background

A bilinear scheme for 3×3 matrix multiplication with r products is a triple of coefficient tensors α, β, γ :

$$M_m = \left(\sum_{a,b} \alpha_{ab}^{(m)} A_{ab} \right) \cdot \left(\sum_{c,d} \beta_{cd}^{(m)} B_{cd} \right), \quad m = 1, \dots, r$$

$$C_{pq} = \sum_m \gamma_{pq}^{(m)} M_m$$

Correctness is equivalent to the **Brent equations**; over GF(2):

$$\bigoplus_{m=1}^r \alpha_{ab}^{(m)} \wedge \beta_{cd}^{(m)} \wedge \gamma_{pq}^{(m)} = \delta_{bc} \delta_{ap} \delta_{dq} \quad \forall (a, b, c, d, p, q)$$

i.e. 729 cubic XOR equations over $27r$ variables ($r=23$: 621 vars; 27 equations have RHS 1 — the “type-3” / delta equations). Any integer scheme reduces mod 2 to a GF(2) scheme; conversely a GF(2) scheme *may* lift to signs ± 1 (HKS observe lifting rarely fails).

Known landscape: $r=23$ achievable (Laderman 1976); best lower bound 19 (Bläser 2003); $r=22$ **open in both directions**. Before HKS (SAT 2019 / J. Symbolic Computation 2021), only 4 inequivalent $\{-1, 0, 1\}$ schemes were known; their local-search campaign found >17,000 more, all published in the Linz database this report checks against.

3 The pipeline

Five stages; each has a tool in `matmul/` and an exact command in §4.

3.1 Native-ANF SLS engine (`src/anf.rs`, binary `anf`)

The Brent system is represented natively as cubic-XOR (“ANF”) constraints over the 621 real variables — no Tseitin auxiliaries. Flipping a variable touches exactly its 81 incident equations; a monomial toggles iff its two partner bits are 1, so incremental evaluation is $O(81)$ per flip (0.4 to 5M flips/s/core measured, ~ 10 M flips/s aggregate over 10 threads). Two policy regimes matter (both WalkSAT-family):

- close repair (many bits frozen at a known scheme): WalkSAT/SKC, noise 0.2, init density 0.25;
- pairing / from-scratch: probSAT, cb 2.5, **init density 0.10** ($\approx$ the free-support density of real completions).

The engine’s structural move is the **tensor closure**: the Brent system is tri-linear, and every equation contains exactly one variable-group of each tensor, so fixing two tensors decomposes the third into 9 independent $81 \times r$ GF(2) linear systems. A consistent single-tensor closure therefore *solves the instance outright*, and each closure call is monotone. It runs as an injected hook every N flips (`--closure-every`).

Baselines on this machine (the Mac mini defined in §1): kissat cannot solve the $r=23$ CNF (unknown at 60s; it needs 41.6s to prove even 2×2 -in-6 UNSAT), matching HKS’s finding that CDCL fails here. yalsat (the HKS solver, v1.0.1 built from source) solves seeded-repair CNFs at their published operating point (fix 414/621 bits: ~ 0.05 to 0.2s) but **times out at 300s at fix=300 and fix=250 — instances the native engine solves in 5ms and 60ms**. On the official challenge-1 instances (hardcoded pairing cores, no streamliners), native SLS + closure solves **8/10** (best two: 0.019s and 0.069s; the 0.069s A-instance solution was additionally confirmed by planting our 621 bits as units into *their* CNF and running kissat: SATISFIABLE). yalsat’s published record on challenge 1 is 5/10 “in a few minutes”; same-machine yalsat probes: A and M time out at 300s, 4-4-4-4-1 solves in 0.62s (native: 0.019s).

3.2 Discovery: neighborhood walk (`matmul/walk.py`)

HKS “method 2”, compounding: maintain a pool (24 verified seed schemes fetched from the Linz DB: 4 classics + 20 spanning 20 rank-pattern directories); repeatedly pick a pool scheme, freeze a random 300 of its 621 bits, let the engine complete the remaining 321 from a random start; canon-dedupe completions (sort the 23 summands); genuinely new schemes join the pool and become seeds themselves. Every accepted scheme is re-verified against the Brent equations by code independent of the search.

The committed run: `--minutes 6 --nfix 300 --runs 8 --rng 7`, single-threaded $\rightarrow$ **138 schemes, distinct after summand sorting, at ~ 3 s/scheme**, accelerating as the pool diversifies.

3.3 Exact equivalence (`matmul/equiv.py`)

De Groote’s symmetry group acts on schemes; “new” must mean “new modulo that group”. Key implementation fact: writing schemes as summands $(A, B, \tilde{C})$ with $\tilde{C} = \gamma^T$, the group action becomes the cyclic sandwich

$$(PAQ^{-1}, QBR^{-1}, R\tilde{C}P^{-1})$$

— and then every constraint “this summand maps to that one” is **linear** in the 27 unknown GF(2) bits of (P, Q, R) . Exact equivalence testing = backtracking over summand matchings (pruned by rank triples) + incremental GF(2) RREF + nullspace enumeration + invertibility + full multiset check; ~ms per pair. An invariant fingerprint (multiset of per-summand sorted rank triples + pair-sum rank triples) prunes almost all pairs first.

Self-tests: 12 random group elements applied to Laderman are (a) still valid schemes and (b) recovered as equivalent-with-witness; Laderman vs Smirnov is refuted. On the 138 finds: **129 distinct classes** (9 pairs were product-reorderings/ G -images of each other).

3.4 Database novelty (matmul/novelty.py, matmul/dbcheck.py via agent)

Two independent layers:

1. **Rank-pattern absence.** The DB’s 302 directory names encode per-summand rank types (legend constraint-solved from 20 known dir $\leftrightarrow$ scheme pairs; letters refine sorted rank triples by slot: a=(1,1,1), b/d/j=(1,1,2), c/g/s=(1,1,3), e/k/m=(1,2,2), f=(1,2,3), n=(2,2,2), w=(2,2,3); unknown letters treated as wildcards — conservative). Rank patterns are G -invariants, so pattern-absence $\Rightarrow$ inequivalence. Control: Laderman’s pattern is absent from all 302 found-scheme dirs — correct (HKS report their finds never reached Laderman’s type).
2. **Full-database exact check.** The complete DB (single schemes.tgz, 43MB = **17,376 .tab files**, byte-validated against live fetches) was converted (γ -transposed), with **0/17,376 parse failures** — every file passes `verify_bits==0` — and 13/13 seed-anchor controls were recovered byte-identically. Per find: fingerprint comparison against all schemes in its compatible dirs, exact `equivalent()` on collisions. **Hardening:** all 17,376 DB schemes were fingerprinted and the surviving finds re-checked ($\times 6$ S_3 variants) against the *whole* database: **zero fingerprint matches anywhere**, making the verdicts independent of the legend.

Verdicts (matmul/novelty_verdicts.csv): **85 of 138 finds are equivalent to DB schemes** (expected — the walk is DB-seeded; witnesses recorded), **53 are new vs the entire database**, and (from §3.3’s audit) inequivalent to the four classics and to each other: 53 finds = 53 classes.

3.5 Integer lifting (matmul/lift.py)

Hypothesis (HKS’s): signs ± 1 exist on the same support. Encoded as **sign-SAT** rather than Gröbner bases: one sign bit per support coefficient; a covering term’s sign is the XOR of its three sign bits; the integer Brent equation with k covering terms and RHS r becomes “exactly $(k - r)/2$ of the k term-bits are 1”; per-product scaling freedom is broken by fixing the first α - and β -support sign of each product. ~2,000-clause CNFs; kissat solves each in milliseconds. Every lift is then verified **exactly over $\mathbb{Z}$** (integer arithmetic, all 729 equations) by code independent of the encoding.

Controls: all 4 classics lift and verify. Result: **53/53 lifted, zero failures**, outputs in `matmul/lifted/*.txt`.

4 Verification chain — check every claim yourself

All commands run from the repo root unless noted; `cd matmul` where shown. Expected wall time in brackets.

```
# 0. build + engine self-tests (incl. closure reconstruction,
#   incremental-vs-scratch equality, Laderman/Strassen embedding)  [~1 min]
cargo build --release --bin anf
cargo test --release --lib anf:      # expect: 7 passed

# 1. generator sanity: Strassen + Laderman verify, instance sizes  [~5 s]
cd matmul && python3 brent.py selftest

# 2. the 53 are valid + distinct-after-sorting                      [~10 s]
grep NEW novelty_verdicts.csv | cut -d, -f1 | sed 's|^|found/|;s|$.bits|' \
```

```

| xargs cat | python3 canon.py 3 3 3 23 /dev/stdin
# expect: 53 schemes read, 0 INVALID, 53 distinct after summand sorting

# 3. exact-equivalence machinery self-test + 53 = 53 classes      [~30 s]
python3 equiv.py selftest
grep NEW novelty_verdicts.csv | cut -d, -f1 | sed 's|^|found/|;s|$.bits|' \
| xargs python3 equiv.py classes
# expect: TOTAL de-Groote classes: 53

# 4. rank-pattern novelty vs the 302 DB dirs (weaker, no download) [~30 s]
python3 novelty.py db_rank_patterns.txt found/walk-00029.bits # etc.

# 5. FULL database check from primary sources                      [~15 min]
python3 dbcheck.py all
# fetch (43 MB schemes.tgz) -> convert all 17,376 .tab with a
# verify_bits==0 gate (expect 0 failures) -> controls (20 db seeds
# byte-identical + 4 classics equivalent) -> check every find against
# ALL DB fingerprints (x6 S3 variants; exact equivalent() on
# collisions; directory-scope-free, so independent of the rank-pattern
# legend). Ends with an automatic cross-check against the committed
# novelty_verdicts.csv: expect "MATCH" and 85 EQUIVALENT / 53 NEW.

# 6. integer lifting + exact Z-verification                       [~1 min]
python3 lift.py seeds/smironov.bits seeds/laderman.bits --outdir /tmp/ctl
grep NEW novelty_verdicts.csv | cut -d, -f1 | sed 's|^|found/|;s|$.bits|' \
| xargs python3 lift.py --outdir /tmp/lift53
# expect: "53 lifted, 0 not +-1-liftable"; each line Z-VERIFIED
# (the Z-check is an assertion: any failure aborts loudly)

# 7. challenge-1 spot-check (their exact CNF file, our scheme)    [~1 min]
git clone https://github.com/marijnheule/matrix-challenges challenges
mkdir -p inst && python3 import_core.py \
challenges/challenge1/MM-23-2-2-2-2-A.cnf inst/core-A.freeze
../target/release/anf 3 3 3 23 --freeze-file inst/core-A.freeze \
--probsat --cb 2.5 --density 0.1 --seconds 60 --threads 10 --seed 3 \
--quiet | grep '^b ' > /tmp/solA.txt
python3 check_their_cnf.py challenges/challenge1/MM-23-2-2-2-2-A.cnf \
/tmp/solA.txt
# expect: "validated type-3 pairing (transpose_gamma=True)" on import,
# then: SATISFIED by our scheme
# (challenges/ and inst/ are gitignored -- external data + derived)

```

What is and isn't deterministic. Verification of the committed 53 (steps 0 to 6) is deterministic. *Discovery* (walk.py) and the challenge-1 solves are stochastic and timing-dependent: rerunning reproduces the *phenomena* (similar yields/times) but not bit-identical artifacts. kissat may return different sign models in step 6 across versions — any model it returns is then $\mathbb{Z}$ -verified, which is the claim that matters.

5 The 53 schemes

Files: mod-2 bit-vectors `matmul/found/walk-*.bits` (the 53 names are the NEW rows of `matmul/novelty_verdicts.csv`); signed integer forms `matmul/lifted/walk-*.txt`. Support = number of nonzero coefficients; for a $3 \times 3 \times 23$ scheme the naive addition count is exactly support – 55. Ours: support 149 to 164 (median 154) = **94 to 109 naive additions**; for reference Laderman = 153/98, Smirnov = 139/84, DB minimum = 139 (support percentiles of the full DB: $p_1 = 146$, median = 159 — our sparsest sits at the 3rd percentile; 22 of the 53 need fewer naive additions than Laderman, none beats the DB's sparsest). Rank-type multiset = per-summand sorted (rank α , rank β , rank γ) with multiplicities — the invariant that separates 51 of the 53 from the whole DB at the coarsest level.

Example (walk-00029, lifted; full file in `matmul/lifted/`):

```

M1 = (+a11 +a32) . (+b11 -b21 +b22)      C11 = +M1 +M10 -M12 +M13 +M17
M2 = (+a21 +a32) . (+b11 -b21 -b23)      C12 = +M1 -M4 -M12 +M13 +M20
... (23 products)                        ... (9 outputs)

```

Full table (53 rows)

scheme	support	rank-type multiset
walk-00028	157	111×12 112×2 113×3 122×2 222×4
walk-00029	156	111×11 112×3 113×3 122×2 222×4
walk-00034	157	111×12 112×2 113×3 122×2 222×4
walk-00035	150	111×14 112×3 113×1 122×1 222×4
walk-00036	151	111×15 112×2 113×1 122×1 222×4
walk-00039	163	111×10 112×7 122×2 222×4
walk-00040	160	111×10 112×7 122×2 222×4
walk-00046	149	111×13 112×6 222×4
walk-00047	153	111×16 112×3 222×4
walk-00048	158	111×16 112×3 222×4
walk-00055	157	111×12 112×5 122×2 222×4
walk-00056	157	111×14 112×3 113×1 122×1 222×4
walk-00058	160	111×10 112×7 122×2 222×4
walk-00060	155	111×8 112×8 113×1 122×2 222×4
walk-00063	155	111×14 112×5 222×4
walk-00064	155	111×15 112×4 222×4
walk-00066	151	111×9 112×8 113×1 122×1 222×4
walk-00072	151	111×12 112×3 113×3 122×1 222×4
walk-00075	154	111×8 112×6 113×4 122×1 222×4
walk-00076	152	111×13 112×6 222×4
walk-00077	150	111×15 112×4 222×4
walk-00081	150	111×14 112×3 113×1 122×1 222×4
walk-00082	152	111×13 112×6 222×4
walk-00083	152	111×13 112×6 222×4
walk-00084	149	111×13 112×6 222×4
walk-00085	153	111×16 112×3 222×4
walk-00091	156	111×10 112×4 113×2 122×3 222×4
walk-00092	153	111×9 112×7 113×1 122×2 222×4
walk-00093	153	111×8 112×9 113×1 122×1 222×4
walk-00094	151	111×9 112×5 113×4 122×1 222×4
walk-00100	160	111×11 112×6 122×2 222×4
walk-00105	151	111×11 112×8 222×4
walk-00106	161	111×9 112×5 113×3 122×2 222×4
walk-00107	149	111×10 112×5 113×3 122×1 222×4
walk-00108	158	111×11 112×3 113×3 122×2 222×4
walk-00115	154	111×14 112×3 113×1 122×1 222×4
walk-00116	155	111×14 112×2 113×1 122×2 222×4
walk-00120	152	111×14 112×3 113×1 122×1 222×4
walk-00121	164	111×8 112×5 113×4 122×2 222×4
walk-00122	163	111×9 112×5 113×3 122×2 222×4
walk-00125	151	111×13 112×4 113×1 122×1 222×4
walk-00127	155	111×15 112×4 222×4
walk-00136	154	111×13 112×6 222×4
walk-00138	154	111×11 112×4 113×3 122×1 222×4
walk-00141	160	111×12 112×2 113×3 122×2 222×4
walk-00143	152	111×13 112×4 113×1 122×1 222×4
walk-00144	150	111×13 112×2 113×3 122×1 222×4
walk-00145	149	111×12 112×3 113×3 122×1 222×4
walk-00149	156	111×14 112×3 113×1 122×1 222×4
walk-00150	159	111×14 112×3 113×1 122×1 222×4
walk-00151	150	111×14 112×3 113×1 122×1 222×4
walk-00157	151	111×11 112×8 222×4
walk-00159	150	111×13 112×2 113×3 122×1 222×4

(Note: the multiset shown is the coarse invariant; several schemes share it — they are separated by the

finer pair-sum fingerprint and/or exact checks. All 53 are pairwise inequivalent.)

6 Secondary results

- **Challenge 1 (HKS): 8/10 official instances solved** (pairing cores, no streamliners; yalsat’s record 5/10). The two holdouts (M, 2-2-2-4-A) floor at best 3/729 after 600s×10 threads.
- **Path-space SLS (connection-method local search): built, verified correct, measured NEGATIVE** at equal budget vs assignment-space SLS (orders of magnitude, all regimes). Diagnosis: the connection objective collapses (a conflicted variable hides force1×force0 repairs) and the Brent matrix’s sharing is dense (81 equations/var) — path rerouting is myopic exactly where an assignment flip re-evaluates everything at once. `matmul/pathsls.py`, `doc/matmul_plan.md` R3c.
- **$r=22$ (challenge 4): open, probed.** Plain native attacks floor at 8/729. Drop-a-product repair floors at 1/729 — but the finisher (`matmul/finisher22.py`) proved every such floor-1 state violates exactly the type-3 equation whose sole cover was dropped, is rigid to radius 3 ($\sim 1.3\text{M}$ flip-sets), and fails per-tensor closure by exactly 1 row each: the floor-1 shell is a *seeding artifact*, not evidence about $r=22$. Campaign infrastructure: `matmul/campaign22.py` (checkpointed; wave 1 = 210 attacks logged).

7 Caveats and scope

- *Re-checked 2026-07-04:* the 53 were additionally tested against Perminov’s scheme collection (<https://github.com/dronperminov/FastMatrixMultiplication>, the source of the Dec-2025 additive-record schemes; 7 distinct rank-23 schemes, all themselves HKS-DB classes) — **zero matches; all 53 remain new**, with positive controls firing.
- “New” = inequivalent under de Groote symmetry to the 17,376 schemes in the Linz database snapshot (`schemes.tgz`, Last-Modified 2020-08-07) and the 4 classics. The database is the comprehensive public record of $\{-1, 0, 1\}^{3 \times 3 \times 23}$ schemes; we make no claim about unpublished or post-2020 private collections.
- Discovery-effort comparisons with HKS are indicative, not controlled: we *start from* their published schemes as seeds, they started from 4; their campaign also had to invent the methods. The like-for-like comparison is our seeded-repair and challenge-1 numbers vs yalsat on the same machine (§3.1). Normalized anyway, for scale: SAT 2019 reports $>13,000$ mutually inequivalent schemes for ≈ 35 CPU-years — ≈ 370 schemes per CPU-year on 2019 server cores, or ≈ 750 to $1,100$ per CPU-year after crediting modern cores (M4-class single-thread is roughly 2 to $3 \times$ a 2019 cluster Xeon’s). Our 53 cost ≈ 46 single-core minutes end-to-end: $\approx 6 \times 10^5$ schemes per CPU-year, roughly 500 to $800 \times$ the speed-adjusted HKS rate. The gap persists in class currency: the full 17,376-scheme database spans 302 de-Groote rank-type patterns ($\lesssim 9$ patterns per CPU-year), while our later flip campaigns (companion notes) added 81 new patterns for ≈ 140 CPU-hours ($\approx 5,000$ per CPU-year, ≈ 200 to $300 \times$ adjusted). The warm start is doing real work in these ratios — which is this report’s thesis: the database turns discovery from search into traversal. A 46-minute controlled pilot replaces “indicative” with “measured” at the start of the curve: seeding `walk.py` with *only the four classics* (HKS’s actual 2019 starting position, database withheld) produced 31 schemes = 29 new de Groote classes in 46 single-core minutes — 122s/scheme cold, 86s/scheme once the pool reached ~ 35 , every find inside existing DB rank patterns (i.e., database rediscovery: the database is the classics’ method-2 neighborhood). That is $\approx 3 \times 10^5$ inequivalent schemes per CPU-year from their own starting position, ≈ 300 to $450 \times$ the speed-adjusted HKS average, before the compounding that carried the warm-pool run to $\sim 3\text{s/scheme}$. The full campaign (6.8 h at 12 threads, ≈ 82 CPU-h) then measured the saturation the pilot could only warn about: archive 31 $\rightarrow$ 618 schemes = 385 de Groote classes, per-half-hour yield decaying 99 $\rightarrow$ 9; the decay fits rate $\propto (N - S)$ with $\hat{N} \approx 715$, i.e. *pure method-2 neighborhood walking from the classics saturates at ≈ 700 schemes* — regenerating the 13,000-scheme corpus requires the diversity injections HKS’s two-method design provided (streamlined cold solves; in our machinery, flip/storm moves or lower-`nfix` jumps). Measured rates: $\approx 63,000$ schemes/CPU-year averaged to saturation, ≈ 15 to 20,000 marginal at saturation, versus

their 371 whole-campaign average. Full-regeneration **estimates** (model-dependent: assume diversity injection sustains between our saturation-marginal and averaged rates): ≈ 0.2 to 0.9 CPU-years for $13,000 - 40$ to $175 \times$ **less compute than the original 35 CPU-years** unadjusted, ≈ 13 to $90 \times$ after the 2 to $3 \times$ hardware credit — and ≈ 0.3 to 1.2 CPU-years for the full $17,376$ -scheme database (no HKS cost is published for the DB’s post- $13,000$ growth). The measured objects are the discovery curve and the ≈ 700 -scheme method-2 basin; the ratios inherit their assumptions (artifacts: `matmul/replica4/`).

- The de-Groote checker, DB converter, and lift verifier are our code; their self-tests and controls are listed in §4. The two strongest external anchors: our A-instance solution satisfies *HKS’s own CNF* under kissat, and the DB fingerprint hardening uses only rank arithmetic + our exact checker with published inputs.
- $51/53$ schemes have four rank- $(2,2,2)$ summands (like most HKS finds); none matches Laderman’s four-quadruple core type.

Acknowledgments

This work was carried out in an extended interactive collaboration with Claude (Anthropic; the Fable 5 and Opus 4.8 models), which implemented the repository’s search tooling, exact minimizers, the Lean formalization, and much of this text under the author’s direction and review. Every computational claim is mechanically checkable by the commands in the reproduction section; the results rest on those independent verifications rather than on trust in either the author or the tools.

8 Pointers

- Repository: <https://github.com/gsidebottom/logic>.
- Plan/lab-notebook: `doc/matmul_plan.md` (every measurement, incl. negatives and retractions).
- Engine: `src/anf.rs`, `src/bin/anf.rs`.
- Tools (all in `matmul/`): `brent.py`, `sls.py`, `walk.py`, `canon.py`, `equiv.py`, `novelty.py`, `lift.py`, `import_core.py`, `check_their_cnf.py`, `dbcheck.py`, `campaign22.py`, `drop22.py`, `finisher22.py`, `pathsls.py`.
- Everything needed for §4 is in-repo except three things fetched from their public sources by the commands shown: the DB archive (`dbcheck.py fetch`), the matrix-challenges clone (step 7), and yalsat (<https://github.com/arminbiere/yalsat>) for the baseline timings.
- HKS: *Local Search for Fast Matrix Multiplication* (SAT 2019, arXiv:1903.11391); *New ways to multiply 3×3 -matrices* (J. Symbolic Computation 104, 2021).
- Scheme database ($17,376$ schemes; no GitHub mirror — the related repo <https://github.com/marijnheule/matrix-challenges> holds the SAT challenges, not the scheme corpus): <http://www.algebra.uni-linz.ac.at/research/matrix-multiplication/> (the `www.` host and plain http are required; https serves a self-signed cert). Snapshot pinned by content: `schemes.tgz`, $43,134,227$ bytes, Last-Modified 2020-08-07, sha256 `4bc8132644504a917e3c076f64df8e6619fb67c55670179853ae5fdb1583074f`. A permanent, content-verified archival mirror of that exact snapshot is deposited at <https://doi.org/10.5281/zenodo.21209925>.
- Challenges: <https://github.com/marijnheule/matrix-challenges>.